

# COMMAND PATTERN

Behavioural Design Pattern

# Problem Statement

Imagine you're building a restaurant management system. Customers can order different food items like **Burger**, **Pizza**, and **Pasta**.

**Without Command Pattern**, the Waiter class would directly call the Chef's cooking methods based on what the customer orders.

This creates tight coupling between the Waiter and the Chef.

The Waiter would need to know:

- What items the Chef can cook
- Exactly which method to call for each item
- How to handle each specific food type

# Command Pattern Structure

The Three Main Components:

- **Command (Order):** This is the interface that all food orders must follow. It defines the *execute()* method that every order type must implement. Think of it as the order slip template – every order slip has the same basic structure.
- **Invoker (Waiter):** This is the one who takes the order and passes it to the kitchen. The Waiter doesn't need to know what's in the order. The Waiter sends the command.
- **Receiver (Chef):** This is the one who actually does the work. The Chef receives the order and performs the cooking operation. The Chef performs the actual task.

# UML Diagram

